DHVANI — Breadboard Build

STM32U575ZI-Q + DAC0800 + TL074 · v2, replaces the DAC904 / OPA365 / perfboard plan

Work through this in order. Do not skip ahead. There are three hard gates. If you are not past a gate by its hour, take the fallback rather than pushing on — pushing past a failed gate is how 24 hours becomes 40.

Plan four hours of sleep around hour 15. Hours 20–24 without it produce bugs you will spend hour 25 finding.

What changed from v1

Item	v1 (old PDF)	v2 (this PDF)
DAC	DAC904 — 14-bit, needs a clock pin	**DAC0800** — 8-bit, no clock pin, output follows the data pins
Op-amp	OPA365 × 3	**TL074** — one chip, four amps: I→V, filter, buffer, one spare
Board	Perfboard	**Breadboard**
Supply	3.3 V only	**+5 V** (Nucleo) and **−9 V** (9 V battery) — the DAC0800 needs a negative rail
Data pins	12 pins, PE0–PE11	**8 pins, PE0–PE7** (PE8–PE11 stay configured, left unconnected)
Sample values	12-bit (0–4095, centre 2048)	**8-bit (0–255, centre 128)**
DMA mode	Circular	**One ping per DMA transfer** (normal mode) — see note below

The architecture

```
U575 PE0-PE7 ──► DAC0800 ──► TL074 A ──► TL074 B ──► TL074 C ──► scope (51 Ω)
  (8 bits)      (current)    (I → V)     (filter)    (buffer)  └► buzzer (1 kΩ)

TIM1 update event ──► GPDMA1 ──► one byte from RAM into GPIOE->ODR per sample
Turbidity / TDS ──► ADC1 ──► firmware picks the next chirp band
```

A hardware timer triggers DMA. DMA copies one byte from RAM straight into the port register, so all eight data bits change at once. The CPU is not involved per sample, and it sleeps (`__WFI`) while each ping plays. That is the brief's requirement, and it is what makes this work.

> **Why one ping per transfer, not circular mode:** a sonar transmitter sends bursts (pings) and then goes quiet, so a single DMA pass per ping is the natural fit. It also avoids a trap: on the U5 the GPDMA only does circular transfers through linked lists, which is much harder to set up.
>
> **Byte writes:** DMA writes a single byte to the bottom of GPIOE->ODR, so it only ever touches PE0–PE7. PE8–PE15 are never overwritten.

HOURS 0–4 · BRING-UP (already done — confirm each line)

Setting	Value	Done?
System clock	160 MHz (MSIS 48 MHz ÷3 ×10 ÷1), no red boxes	☐
Power	Voltage Scale 1, SMPS	☐
ICACHE	1-way (direct mapped)	☐
LED	`BSP_LED_Toggle(LED_GREEN)` blinks LD1	☐
Data pins	PE0–PE11 GPIO_Output, speed Very High, labels DAC_B0...B11	☐
CLK / trigger pin	PA8 = TIM1_CH1 (not PE9)	☐
TIM1	Prescaler 0, Counter Period 159, Pulse 80, auto-reload preload Enable	☐
PA8 test	About 1.6 V on a good meter (half of 3V3)	☐

PA8 is no longer the DAC's clock — the DAC0800 has no clock pin. Keep TIM1 anyway: its update event is what triggers the DMA, and PA8 becomes a handy scope trigger.

> **GATE 1 — hour 4**
>
> LED blinking. PE0–PE7 set as outputs. TIM1 running at 1 MHz. If the clock configuration fights you, stop and ask for help.

HOURS 4–9 · BREADBOARD AND DC TEST

4.1 Parts

Part	Qty	Used for
DAC0800 (16-pin DIP)	1	The DAC
TL074 (14-pin DIP)	1	All op-amp stages
3.3 kΩ	2	DAC reference, pins 14 and 15
2.7 kΩ	1	I→V feedback
1.5 kΩ	2	Filter (hour 19)
1 kΩ, 51 Ω	1 each	Buzzer tap, scope output (hour 19)
10 nF	1	DAC compensation, pin 16
100 nF	6	Decoupling
150 pF	2	Filter (hour 19)
22 pF	1	Optional, across the 2.7 kΩ
9 V battery + clip	1	−9 V rail
Passive piezo buzzer	1	Audible demo
Solid-core jumper wire	many	Everything

4.2 Breadboard rules

- **Label your rails with tape:** GND, +5V, 3V3, −9V. Mixing up +5 V and −9 V is the one mistake that can kill a chip.
- **Check for split rails.** A gap in the red/blue line means the rail is broken in the middle — bridge it with a short wire.
- **Capacitors at the chip.** Every 100 nF goes right next to the pin it serves.
- **Short op-amp wiring.** Put the DAC0800 and TL074 side by side. Keep the DAC pin 4 → TL074 pin 2 wire and the 2.7 kΩ as short as possible.

- **Solid-core wire, pressed in firmly.** Most breadboard faults are loose contacts — if a reading jumps, wiggle wires first.
- **Battery goes on last,** after the beep checks.
- Chips go across the middle gap, notch facing left: pin 1 is bottom-left, pins count along the bottom and back along the top.

4.3 Data wires — Nucleo to DAC0800

Nucleo pin	Where it is	DAC0800 pin	Bit
PE7	CN10 pin 20	5	B1 — MSB
PE6	CN9 pin 20	6	B2
PE5	CN9 pin 18	7	B3
PE4	CN9 pin 16	8	B4
PE3	CN9 pin 22	9	B5
PE2	CN9 pin 14	10	B6
PE1	**CN11 pin 61** (morpho only — not on Zio)	11	B7
PE0	CN10 pin 33	12	B8 — LSB

PE7 goes to pin 5, not pin 12. On the DAC0800, pin 5 is the most significant bit.

4.4 Power

From	To
CN8 pin 11 (GND)	Breadboard blue rail = GND everywhere below
CN8 pin 9 (5V)	+5V rail → DAC0800 pin 13 and TL074 pin 4
CN8 pin 7 (3V3)	3V3 row → DAC0800 reference resistor only
9 V battery red (+)	**GND rail**
9 V battery black (−)	−9V rail → DAC0800 pin 3 and TL074 pin 11

4.5 DAC0800 — the remaining pins

Pin	Name	Connect to
1	VLC (logic threshold)	GND — sets a ~1.4 V threshold, so 3.3 V GPIO drives it directly
2	IO-bar (complementary out)	GND
3	V−	−9V rail, plus 100 nF to GND
4	IO (main output)	TL074 pin 2
13	V+	+5V rail, plus 100 nF to GND
14	VREF+	3.3 kΩ to the 3V3 row (sets ~1 mA reference current)
15	VREF−	3.3 kΩ to GND
16	COMP	10 nF to pin 3

4.6 TL074 — power, amplifier A, and parking the rest

TL074 pin	Connect to	Why
4	+5V rail, plus 100 nF to GND	Positive supply
11	−9V rail, plus 100 nF to GND	Negative supply
3	GND	Amp A + input
2	DAC0800 pin 4	Amp A − input (virtual ground)
1 ↔ 2	2.7 kΩ (optional 22 pF in parallel)	Feedback — sets 0 to ~2.7 V output
1	**Test point**	Amp A output
5 → GND, 6 ↔ 7	Park amp B for now	Unused amps can oscillate
10 → GND, 9 ↔ 8	Park amp C for now	
12 → GND, 13 ↔ 14	Park amp D permanently	

The DAC0800 *sinks* current, so pulling current out of amp A's − input drives its output positive: $V_{out} \approx I_{out} \times 2.7\ k\Omega$.

4.7 Checks before you connect USB

- Replace the multimeter battery first. A weak battery made it read about 35% high (3.3 V showed as 4.5 V).
- Beep mode, USB unplugged, battery off: **+5V to GND, 3V3 to GND, −9V to GND and TL074 pin 4 to pin 11 must NOT beep.**
- Each PE wire beeps to its DAC pin. Neighbouring DAC pins do not beep to each other.
- Clip on the battery. **DAC0800 pin 3 must read about −9 V** — with the minus sign. +9 V means the battery is reversed; fix it before USB.
- TL074 pin 11 reads the same −9 V.

4.8 The DC test

In `main.c`. Remove the blink lines from USER CODE 3.

```
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);   // not needed by the DAC0800, harmless
static const uint8_t test[3] = { 0x00, 0x80, 0xFF };
/* USER CODE END 2 */

  /* USER CODE BEGIN 3 */
  for (int i = 0; i < 3; i++)
  {
    GPIOE->ODR = test[i];        // PE0-PE7 change together
    BSP_LED_Toggle(LED_GREEN);   // LED flips on each step
    HAL_Delay(3000);
  }
}
/* USER CODE END 3 */
```

Put the meter on TL074 pin 1 against GND.

GATE 2 — hour 9 — THE CRITICAL ONE

Three voltages cycling every three seconds: **~0 V → ~1.35 V → ~2.7 V**.

The middle reading must be **half** the top one. A few tens of millivolts, or a tiny negative reading, at the bottom is normal for the TL074.

If these are wrong, stop. Nothing downstream will work and you will waste hours chasing it in firmware.

Symptom	Cause
Always 0 V	No reference current (pin 14 resistor or 3V3 row), V− not connected, or DAC pin 4 not wired to TL074 pin 2
Stuck near +3.5 V	2.7 kΩ feedback missing, or DAC pins 2 and 4 swapped
Steps in the wrong proportions	Data wires out of order — PE7 must go to pin 5
Reading jumps around	Loose wire, missing 10 nF on pin 16, or missing 100 nF decoupling
Chip gets hot	Unplug now. Check battery polarity and the +5V / −9V rails

HOURS 9–15 · DMA STREAMING

This is the real work. Everything else is assembly.

9.1 Configure GPDMA in CubeMX

System Core → GPDMA1 → Channel 0 → **Standard Request Mode**. (In some CubeMX versions you add the request from TIM1 → DMA Settings → Add instead — the values are the same.)

Setting	Value
Request	TIM1_UP
Direction	Memory to Peripheral
Source address increment	Enabled
Destination address increment	Disabled
Source data width	Byte
Destination data width	Byte
NVIC	GPDMA1 Channel 0 global interrupt — **Enabled**

Generate code. Check the handle name CubeMX created — search `main.c` for `DMA_HandleTypeDef`. The code below assumes `handle_GPDMA1_Channel0`.

9.2 Stream one ping

```
/* USER CODE BEGIN Includes */
#include <math.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
#define N 20000                      // 20 ms of samples at 1 MSps
static uint8_t wave[N];
volatile uint8_t ping_done = 1;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
static float sample_rate(void)      // always matches the timer, so frequencies stay right
{
  return 160000000.0f / (float)(TIM1->ARR + 1);
}

static void on_ping_done(DMA_HandleTypeDef *hdma)
{
  __HAL_TIM_DISABLE_DMA(&htim1, TIM_DMA_UPDATE);
  ping_done = 1;
}

static void fire_ping(void)
{
  ping_done = 0;
  handle_GPDMA1_Channel0.XferCpltCallback = on_ping_done;
  HAL_DMA_Start_IT(&handle_GPDMA1_Channel0, (uint32_t)wave,
                   (uint32_t)&GPIOE->ODR, N);          // N bytes
  __HAL_TIM_ENABLE_DMA(&htim1, TIM_DMA_UPDATE);        // timer now paces the DMA
}

static void build_tone(float f)     // Hann-windowed tone burst
{
  float fs = sample_rate();
  for (int i = 0; i < N; i++)
  {
    float w = 0.5f * (1.0f - cosf(2.0f * 3.14159265f * i / (N - 1)));
    wave[i] = (uint8_t)(128.0f + 120.0f * w * sinf(2.0f * 3.14159265f * f * i / fs) + 0.5f);
  }
}
/* USER CODE END 0 */
```

```
  /* USER CODE BEGIN 2 */
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);   // timer runs; PA8 = scope trigger
  build_tone(10000.0f);                        // 10 kHz: audible and easy to see
  /* USER CODE END 2 */

    /* USER CODE BEGIN 3 */
    fire_ping();
    BSP_LED_Toggle(LED_GREEN);
    while (!ping_done) { __WFI(); }            // CPU sleeps while DMA plays the ping
    HAL_Delay(200);                            // quiet time between pings
  }
  /* USER CODE END 3 */
```

Check: scope on TL074 pin 1 at 5 ms/div — a 20 ms burst that swells and fades (the Hann window) every ~220 ms. No scope yet? Clip the buzzer through 1 kΩ to TL074 pin 1 and you will hear short beeps.

9.3 Find your ceiling

Add these two lines at the top of USER CODE 2, and step the value down one run at a time:

```
  __HAL_TIM_SET_AUTORELOAD(&htim1, 159);                        // try 159, 79, 39, 19, 15
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, (159 + 1) / 2);   // keep PA8 at 50 %
```

Counter Period	Sample rate	Ping length (N = 20000)
159	1 MSps	20 ms
79	2 MSps	10 ms
39	4 MSps	5 ms
19	8 MSps	2.5 ms
15	10 MSps	2 ms

Because `sample_rate()` reads the timer, the tone stays at 10 kHz at every setting. Stop when the waveform degrades — missing steps, jitter, or a lower amplitude. The DAC0800 needs about 100 ns to settle, so expect trouble above roughly 5–8 MSps even if the DMA keeps up. **Write the last clean number down.** It is a slide: "the U575 sustained X MSps with the CPU asleep."

> **GATE 3 — hour 15**
>
> Repeating bursts on the scope (or beeps on the buzzer), with the CPU sleeping during each ping.
>
> **Fallback** if DMA will not cooperate: a timer interrupt writing `GPIOE->ODR` directly. Lower ceiling, but it works — and you state the limitation honestly rather than having nothing.

SLEEP — hours 15–19

Four hours. Non-negotiable.

HOURS 19–21 · CHIRP AND ANALOG CHAIN

19.1 The chirp function

Add to USER CODE 0, next to `build_tone`:

```
static void build_chirp(float f0, float f1)   // Hann-windowed linear chirp, f0 -> f1
{
  float fs = sample_rate();
  float T  = N / fs;
  float k  = (f1 - f0) / T;
  for (int i = 0; i < N; i++)
  {
    float t  = i / fs;
    float ph = 2.0f * 3.14159265f * (f0 * t + 0.5f * k * t * t);
    float w  = 0.5f * (1.0f - cosf(2.0f * 3.14159265f * i / (N - 1)));
    wave[i]  = (uint8_t)(128.0f + 120.0f * w * sinf(ph) + 0.5f);
  }
}
```

Test it: replace `build_tone(10000.0f)` with `build_chirp(2000.0f, 15000.0f)`. On the scope the cycles tighten across the burst; on the buzzer you hear a rising "chirp".

19.2 Filter (amp B) and buffer (amp C)

Remove the parking wires on amps B and C first. Leave amp D parked.

```
 TL074 pin 1 ──[1.5k]──┬──[1.5k]──┬──── pin 5 (+B)        pin 6 ── pin 7  (B follower)
  (amp A out)          │          │
                     [150pF]    [150pF]
                       │          │
                     pin 7        GND

 pin 7 (B out) ──── pin 10 (+C)        pin 9 ── pin 8  (C follower)

 pin 8 (C out) ──[51Ω]──── scope / BNC
 pin 8 (C out) ──[1kΩ]──── buzzer +      buzzer − ── GND
```

This is a unity-gain Sallen-Key low-pass with a cut-off near 700 kHz. It smooths the DAC steps for the ultrasonic band and changes nothing you can hear. Use a **passive piezo** buzzer; a magnetic buzzer needs more current than the TL074 can give.

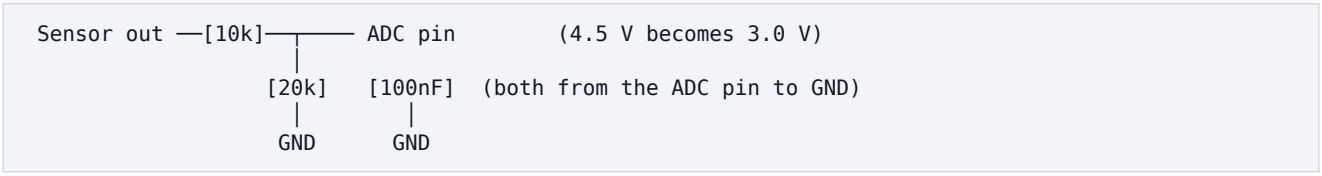
HOURS 21–23 · SENSORS AND ADAPTATION

21.1 Configure ADC1 in CubeMX

Setting	Value
Channels	IN8 = PA3 (Zio A0, CN9 pin 1) — turbidity; IN4 = PC3 (Zio A2, CN9 pin 5) — TDS
Resolution	12 bits (the U5 defaults to 14 — change it)
Scan Conversion Mode	Enabled, Number of Conversion 2, Rank 1 = IN8, Rank 2 = IN4
End Of Conversion Selection	End of single conversion
Sampling time	A long one for both ranks (e.g. 68 cycles or more) — sensor outputs are high-impedance
Clock	If the Clock tab turns red on the ADC, set the ADC clock mux to HSI (16 MHz)

21.2 Wire the sensors

Power both sensors from +5V (CN8 pin 9) and GND. **Measure each sensor's output with the meter first.** Turbidity modules often reach ~4.5 V and need the divider; TDS modules usually stay below 3.3 V and connect directly.



21.3 The adaptation loop

```
/* USER CODE BEGIN PV */      (add below the existing lines)
#define AUDIBLE 1             // 1 = buzzer demo band, 0 = ultrasonic band for the scope
uint32_t turbidity = 0, tds = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */       (add below the existing functions)
static void read_sensors(void)
{
  HAL_ADC_Start(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, 10);  turbidity = HAL_ADC_GetValue(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, 10);  tds       = HAL_ADC_GetValue(&hadc1);
  HAL_ADC_Stop(&hadc1);
}
/* USER CODE END 0 */

  /* USER CODE BEGIN 2 */     (add as the first line)
  HAL_ADCEx_Calibration_Start(&hadc1, ADC_CALIB_OFFSET, ADC_SINGLE_ENDED);
```

```
    /* USER CODE BEGIN 3 */   (replaces the 9.2 loop)
    static uint32_t avg = 0;
    read_sensors();
    avg = (avg * 3 + turbidity) / 4;              // smooth, 0..4095

#if AUDIBLE
    float f0 = 2000.0f,   f_hi = 5000.0f   + avg * 3.0f;    // up to ~17 kHz
#else
    float f0 = 100000.0f, f_hi = 120000.0f + avg * 32.0f;   // up to ~250 kHz
#endif
    float f_max = 0.25f * sample_rate();          // stay below fs/4 for a clean sine
    if (f_hi > f_max) f_hi = f_max;

    build_chirp(f0, f_hi);                        // ~10-30 ms of CPU, between pings only
    fire_ping();
    BSP_LED_Toggle(LED_GREEN);
    while (!ping_done) { __WFI(); }               // CPU asleep while the ping plays
    HAL_Delay(200);
  }
  /* USER CODE END 3 */
```

21.4 Verify the direction

Move the probe between clear and muddy water. **Pitch drops in muddy water → correct** (muddy water absorbs high frequencies more, so the transmitter moves to a lower band). If the pitch rises instead, your module reads the other way round — replace `avg` with `(4095 - avg)` in the `f_hi` line.

HOURS 23–24 · CAPTURE

	Capture
☐	Scope photo — clear water (ultrasonic mode, `AUDIBLE 0`)
☐	Scope photo — muddy water, same scope settings
☐	Photo of the whole breadboard setup from above
☐	Demo video: probe moving between cups, chirp audible (`AUDIBLE 1`)
☐	Before filming: press every wire in, tape the battery down, two full rehearsals
☐	Numbers written down: max clean sample rate, frequency range reached, CPU asleep during pings, part count

Say it honestly on the slides: 8-bit DAC, so a ~48 dB noise floor — Hann-window sidelobes (about −31 dB) still show clearly above it. Breadboard proof-of-concept; the KiCad PCB is the December step. DAC0800 and TL074 were chosen for availability — the low-power case rests on DMA with the CPU asleep and duty-cycled pings.

THE THREE GATES

Hour	Must have	If not
4	LED blinking, PE0–PE7 outputs, TIM1 at 1 MHz	Clock problem — ask for help
9	TL074 pin 1 steps 0 / 1.35 / 2.7 V	Stop. Fix wiring.
15	DMA pings, CPU sleeping between samples	Fall back to timer interrupt

What to skip

- DS18B20 — an hour of 1-Wire timing for a number on a screen.
- Pressure sensor — same.
- PCB — 10 to 14 days including fabrication. December.

- Moving to perfboard — optional polish only if you finish early.
- Extra modulation types — December work.

None of these change what a judge sees.

When things go wrong

Symptom	Likely cause
Meter readings all too high	Weak meter battery — replace it
No output at all	Battery off or reversed, pin 14 resistor missing, or DAC pin 4 not at TL074 pin 2
Output stuck high	Feedback resistor missing or DAC pins 2/4 swapped
Waveform wrong shape	Data wires out of order — PE7 → pin 5 ... PE0 → pin 12
Nothing streams	GPDMA interrupt not enabled in NVIC, wrong handle name, or TIM1 not started
Staircase, not smooth	Filter capacitors wrong or absent (ultrasonic mode only)
Flat top and bottom	Reduce 120.0f to 100.0f in the waveform functions
Random glitching	Loose breadboard wire or missing decoupling
Frequencies all wrong	Something changed ARR without going through sample_rate() — rebuild the waveform
Sensor does nothing	Measure the voltage at the ADC pin; check the ADC is set to 12-bit

The four things that decide this

1	**Battery polarity.** DAC0800 pin 3 reads −9 V before USB goes in.
2	**Short feedback, caps at the chips.** The 2.7 kΩ and every 100 nF sit right at their pins.
3	**PE7 → pin 5.** The MSB is pin 5, the LSB is pin 12.
4	**Gate 2 passed before any DMA code.**

Start at 4.1. Shout at each gate.